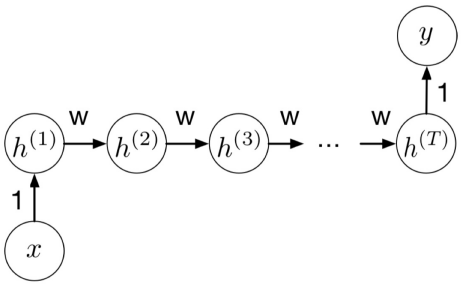
Practice Test 2 for COMS 4776
Neural Networks and Deep Learning
Fall 2025

Name: _____

Uni: _____

1. Consider the following RNN, which has a scalar input at the first time step, makes a scalar prediction at the last time step, and uses a shifted logistic activation function:

$$f(z) = \sigma(z) - 0.5$$



(a) Write the formula for the derivative $\overline{h_t}$ as a function of $\overline{h_{t+1}}$, for $t < T$. (Use z_t to denote the input to the activation function at time t. You may write your answer in terms of σ', i.e., you don't need to explicitly write out the derivative of σ.)

We first note that $\overline{h_t} = \frac{\partial h_{t+1}}{\partial h_t}\overline{h_{t+1}}$. We can compute this partial derivative in two steps. First, $\frac{\partial h_{t+1}}{\partial z_{t+1}} = \sigma'(z_{t+1})$. Then, $\frac{\partial h_{t+1}}{\partial h_t} = \frac{\partial h_{t+1}}{\partial z_{t+1}}\frac{\partial z_{t+1}}{\partial h_t} = \sigma'(z_{t+1})w$. Thus, we have that $\overline{h_t} = w\sigma'(z_{t+1})\overline{h_{t+1}}$.

(b) Suppose the input to the network is $x = 0$. Notice that $h_t = 0$ for all t. Based on your answer to part (a), determine the value α such that if $w < \alpha$, the gradient vanishes, while if $w > \alpha$, the gradient explodes. You may use the fact that $\sigma'(0) = 1/4$.

Using our answer from part (a), we see that $\overline{h_1} = w^{T-1}\sigma'(z_2)...\sigma'(z_T)\overline{h_T}$. For the input $x = 0$, since $z_t = 0$ for all t, we have $\overline{h_1} = w^{T-1}(\frac{1}{4})^{T-1}\overline{h_T} = (\frac{w}{4})^{T-1}\overline{h_T}$. From this expression, we can see that the gradient will explode when $\frac{w}{4} > 1$, and vanish when $\frac{w}{4} < 1$. Thus, the value α representing the 'tipping point' between vanishing and exploding is $\alpha = 4$.

2. Language Models and Transformers

(a) In the encoder/decoder architecture for machine translation, the network reads a sentence and stores all the information in its hidden units. Describe two ways in which a bidirectional RNN encoder differs from this.

Some acceptable differences are listed below.

i. The bidirectional RNN encoder has an RNN that runs forward and an RNN that runs backwards, and the hidden vectors from each RNN are concatenated for each word. This is in contrast to the encoder/decoder architecture where the encoder is a unidirectional RNN that proceeds only from left to right.

ii. The encoder in the encoder/decoder architecture encodes all the sentence information in the right-most hidden vector, while the bidirectional RNN encoder computes a "distributed" representation of the sentence where the sentence is encoded by the entire sequence of vectors.

iii. The bidirectional RNN encoder computes an 'annotation' of each word in the input, which is used by the decoder via an attention mechanism. Precisely, at each time-step an attention score is computed for each of the encoder's annotations, and an attention-weighed sum of the annotations is provided to the decoder as a 'context vector'. In contrast, in the encoder-decoder architecture the decoder uses only the right-most hidden vector from the encoder to begin decoding (there is no attention mechanism).

(b) Explain an advantage of transformers over RNNs, focusing on computational efficiency. Then explain one disadvantage of transformers vs. RNNs in terms of computational efficiency.

Since an RNN performs sequential operations to update its hidden state, the computations cannot be parallelized. By contrast, the attention mechanism in a transformer can be parallelized across tokens. This parallelization can significantly speed up model training and enable scaling to larger datasets.

One disadvantage of transformers vs. RNNs is that they may be slower at one-token-at-a-time decoding during inference. This is because self-attention still

requires $O(n)$ computation for each decoding time-step to compute pairwise interactions with the whole sequence. Since RNNs maintain a single vector representation of the previous context, decoding is constant-time w.r.t sequence length. This is likely to yield speed improvements even if self-attention computation is parallelized.

(c) How do *self-attention* and *cross-attention* differ?

In cross-attention, the keys and values are computed from the representations for one sequence (typically from the encoder), while the queries come from another sequence (typically from the decoder). This operation is useful in encoder-decoder transformer architectures used for example in machine translation. On the other hand, for self-attention the queries, keys, and values all come from the same sequence of token representations, whether it be from an encoder model or decoder model.

3. The standard linear transformer variant linearizes the attention computation as follows:

$$\text{LinAtt}(Q, K, V) = D^{-1}\left(\phi(Q)\left(\phi(K)^\top V\right)\right), \quad D = \text{diag}\left(\phi(Q)\,\phi(K)^\top \mathbf{1}\right),$$

where $\phi(\cdot)$ is a feature map applied element-wise. Several choices of $\phi(\cdot)$ can be used, including a ReLU feature map or one that approximates standard softmax attention.

(a) Assume that the shapes of Q, K, and V are as above. State the dimensions of $\phi(Q)$, $\phi(K)$, $\phi(K)^\top V$, and $\phi(Q)\,(\phi(K)^\top V)$. Since $\phi(\cdot)$ is an element-wise function, the shapes of $\phi(Q)$ and $\phi(K)$ are $n \times d$. The shape of $\phi(K)^\top V$ is $d \times d$, and the shape of $\phi(Q)\,(\phi(K)^\top V)$ is $n \times d$.

(b) Provide the computational complexity of linear attention in terms of n and d.

Computing $\phi(K)^\top V$ costs nd^2 operations. Subsequently computing $\phi(Q)\,(\phi(K)^\top V)$ also costs nd^2 operations. Since D is diagonal, the operations involving D can be computed efficiently and so are dominated by the cost of the above operations. Thus, the computational cost of linear attention is $O(nd^2)$.

(c) Provide the memory complexity of linear attention in terms of n and d. Excluding the inputs and output, computing $\phi(Q)$ and $\phi(K)$ involves $O(nd)$ memory, computing $\phi(K)^\top V$ involves $O(d^2)$ memory, and computing $\phi(Q)\,(\phi(K)^\top V)$ involves another $O(nd)$ memory. Thus, the total memory cost is $O(nd + d^2)$.

(d) Using your results above, explain when linear transformers would be more efficient than the standard self-attention, and why this is advantageous in practice for model training and inference.

Standard self-attention is $O(n^2)$ in both time and space, while the linear transformer is $O(n)$ in both time and space. For long sequences where $n \gg d$, linear transformers are significantly more efficient than self-attention. Practically, this can make model training faster for long sequence lengths, which is useful when you

want to model long-range dependencies in the input. This benefit is especially significant when there is a large amount of training data, such as for language modeling. Inference is also faster especially for long sequence lengths.

4. Adapting Transformers

(a) Given a scenario where you need to adapt a large language model for medical diagnosis tasks with limited computational resources, which adaptation method(s) would you recommend and why?

One answer is using Low-Rank Adaptation (LoRA). LoRA involves finetuning the LLM by adding a low-rank update to the weight matrix by using the equation W' = W + AB, A $\in \mathcal{R}^{dxr}$ and $B \in \mathcal{R}^{rxd}$, where $r \ll d$. LoRA enables the model to specialize for a crucial and complex task like medical diagnosis. Since the number of trainable parameters is drastically reduced, it makes the process resource efficient and meets the constraint of limited resources.

(b) What is In-Context Learning (ICL)? Give an example and explain why it's considered an "inference-time adaptation" method.

In-Context Learning (ICL) is the ability of an LLM to perform new tasks by learning from the input prompt, without updating any model parameters. An example is few-shot translation, where the prompt contains: "Translate to French: Hello → Bonjour, Thank you → Merci, Good Morning →". The model completes the pattern by generating the translation. It is an inference-time adaptation because the model adapts its output during the forward pass (inference) based solely on the context (input), rather than any changes to its parameters.

5. Autoencoders

(a) What is the training objective for a standard autoencoder?

The training objective is the reconstruction error (usually squared error) between the true input and reconstructed input from the bottleneck-layer latent vector.

(b) Describe the two terms in the training objective of a VAE, and how they relate to the overall aim of the model.

The two terms are (1) the reconstruction error between the true input and predicted input from the decoder, and (2) the KL term which encourages the predicted latent distribution $q(\mathbf{z}|\mathbf{x})$ for any input to be close (i.e. low KL divergence) to a simple distribution (typically standard normal).

(c) Consider a trained VAE and a trained GAN. Compare how they are used to generate samples.

For a VAE, a random vector $\mathbf{z}$ can be sampled from the latent space, for instance from the prior $p(\mathbf{z})$, and this can be fed into the decoder network to obtain a sample. For a GAN, a random vector $\mathbf{z}$ can be sampled from the fixed, simple distribution (e.g. spherical Gaussian), and input into the generator to generate a sample.

(d) Specify the Autoencoder formulation that relates closely to PCA, and explain how they differ.

Linear autoencoders with squared error loss are equivalent to a PCA. The eigenvectors of the latent space of such an autoencoder correspond to the directions of maximum variance of the input data distribution.

6. Recall the discriminator's cost function in a GAN:

$$\mathcal{J}_D = \mathbb{E}_{\mathbf{x}\sim\mathcal{D}}[-\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z}}[-\log(1 - D(G(\mathbf{z})))]$$

(a) What do $D(\mathbf{x})$ and $G(\mathbf{z})$ represent?

$\mathbf{z}$ is a random vector sampled from a fixed, simple distribution (e.g. spherical Gaussian). $G(\mathbf{z})$ represents the generator network which maps this vector to an $\mathbf{x}$ in data space (e.g. an image), with the aim of outputting a realistic sample. $D(\mathbf{x})$ is the discriminator network which takes an input $\mathbf{x}$ in data space and tries to figure out its source; it outputs the probability of the input being data rather than produced by the generator network.

(b) Explain why this is a sensible cost function for the discriminator.

The discriminator's job is to estimate whether the input is data or produced by the generator. The cost function captures exactly this by first requiring the predicted probability $D(\mathbf{x})$ to be maximized on inputs from the training dataset (assumed to be sampled from a 'true' underlying data distribution $\mathcal{D}$). And second, requiring $1 - D(G(\mathbf{z}))$ to be maximized, i.e. the predicted probability minimized, for inputs produced by the generator network.

(c) One possible cost function for the generator is the opposite of the discriminator's. Explain why this is sensible.

The generator's objective is to 'fool' the discriminator and make it perform poorly. By inverting the loss function, the generator tries to maximize the discriminator's predicted probability of being data for its generated inputs $G(\mathbf{z})$, and thus drive the loss of the discriminator high.

(d) However, in practice, the generator is usually trained with a different loss function. What cost function do we typically use for the generator?

We typically use the cost function $\mathbb{E}_{\mathbf{z}}[-\log D(G(\mathbf{z}))]$ instead.

(e) What is the reason to use this cost function rather than the one given above?

The problem with the original cost function is *saturation*; when the generated sample is really bad, the discriminator's prediction is close to 0, and in this region the generator's cost is flat and provides a weak gradient signal. The new cost function represents the same conceptual idea but provides strong gradient signals for bad samples.

7. The CycleGAN model

   (a) If the model is successful, what will it be able to do?

   If the model is successful it will be able to change the style of an image from one style to another (e.g. photo to Monet, and back), while preserving the content.

   (b) What data is required to train the model?

   We require two unrelated/unpaired collections of images, one for each style.

   (c) What might go wrong if we eliminate the discriminator terms from the cost function?

   The discriminator ensures that the generated image is in the target style.

   (d) What might go wrong if we eliminate the reconstruction term from the cost function?

   The reconstruction term ensures that the content is preserved, following the mapping to the other style and back to the original style.